



操作系统第四次实验

姓名： 江李阳 王健一 邱泽辉

学号： 202400460042
202400460037
202400460046

专业： 密码科学与技术

日期： 2026 年 6 月 5 日

目录

1 实验目的	2
2 实验环境	2
3 实验原理	2
3.1 FAT	2
3.2 文件的打开与关闭	2
4 任务一	3
4.1 任务内容	3
4.2 任务实现规划	3
4.2.1 常量定义头文件	3
4.3 函数声明	4
4.3.1 文件系统初始化	5
4.3.2 文件目录操作函数	6
4.3.3 文件的打开与关闭	8
4.4 文件读写与删除	10
4.4.1 文件系统入口	16
4.4.2 FAT-16 文件系统使用测试	21
4.5 任务一函数汇总	22
5 任务二	22
5.1 任务内容	22
5.2 任务实现规划	22
5.2.1 头文件常量定义与函数声明	22
5.2.2 系统初始化	24
5.2.3 文件目录操作	26
5.2.4 文件打开与关闭	28
5.3 任务二函数汇总	32
5.3.1 文件读写	32
5.3.2 文件系统入口	33
5.3.3 多进程访问测试	34
6 实验问题	36
6.1 sem_wait 阻塞进程	36
6.2 使用 sem_trywait 修改	36
7 实验结论	36
8 收获与心得	36

1 实验目的

- 任务一: 实现一个简单的类 FAT-16 文件系统, 具有 `mkdir, delete, open, read, write, close` 等功能。
- 任务二: 实现多进程同时访问文件、目录的功能。

2 实验环境

- 硬件环境: 13th Gen Intel(R) Core(TM) i5-13420H (12 CPUs), 2.1GHz 处理器;16384MB RAM 内存。
- 操作系统: Arch Linux(kernel:x86_64 Linux 6.6.87.2-microsoft-standard-WSL2)
- 软件环境: Visual studio code
- 编译器: gcc (GCC) 15.2.1 20260209

3 实验原理

3.1 文件系统布局

文件系统存放在磁盘中, 多数磁盘划分为一个或多个分区, 每个分区有一个独立的文件系统。磁盘分区和文件系统的基本组成如下:

- 主引导记录 (Master Boot Record, MBR): 位于磁盘的 0 号扇区, 用来引导计算机, 其后面是分区表, 该表给出每个分区的起始和结束地址。
- 引导块 (boot block): MBR 执行引导块中的程序后, 该程序负责启动该分区中的操作系统。
- 超级块 (super block): 包含文件系统的所有关键信息, 在计算机启动时会被读入内存。超级块中包含分区的块的数量、块的大小、空闲分区的数量和指针、空闲 FCB 数量和 FCB 指针。
- 文件系统空闲块信息: 可以使用位示图或指针链接的形式给出; 后面跟着一组 i 节点 (每个文件对应一个节点); 接着是根目录, 存放在文件系统目录树的根部; 最后是所有其他文件的目录和文件。

3.2 FAT

显式链接是文件分配物理块的一种方式 (类似静态链表), 其中将链接文件所有块的指针显示地存入内存的一张表中, 即文件分配表 (FAT, File Allocation Table), 每个表项存放链接指针, 即下一盘块号。因此只要将文件的第一个盘块号存放在 FCB 中, 后续盘块号都可以通过查 FAT 表来获取。因此 FAT 的表项与全部磁盘块一一对应, 可以使用一个特殊数字如 -1 表示空闲块, -2 表示文件结束块。FAT 不仅记录了文件各块之间的前后链接关系, 还标记了空闲块信息, 因此可以用来实现文件管理如分配空闲磁盘块等。FAT 的优点是该表在系统启动时读入内存, 因此查找记录的过程是在内存中进行的, 检索速度很快, 减少了访问磁盘的次数。缺点是每次都要将整个 FAT 读入内存, 这会占用较大的内存空间。

3.3 文件的打开与关闭

当用户对文件进行操作时, 每次都需要从检索目录开始, 这将大大降低文件操作的效率。因此文件在使用前需要通过系统调用 `open` 被显式地打开。操作系统维护一张包含所有文件信息的表 (打开文件表)。这里”打开”的含义是指调用 `open` 根据文件名搜索目录, 将指明文件的属性从外存复制到内存打开文件表的一个表目中, 并将该表目的编号 (索引, 也即文件描述符) 返回给用户。当用户再次向系统发出文件操作请求时, 通过索引在打开文件表中查到文件信息, 从而节省再次搜索目录的开销。当文件不再使用时, 利用系统调用 `close` 关闭文件, 实质即为操作系统从打开文件表中删除该文件的条目。

操作系统中存在多个进程可以同时打开文件时, 采用两级表: 整个系统表和每个进程表。整个系统的打开文件表包含 FCB 的副本级其他信息, 每个进程的打开文件表包含指向系统表中适当条目的指针。因此系统打开文件表为每个文件关联一个打开计数器 (Open Count), 以记录有多少个进程打开了该文件。每个关闭操作 `close` 使 `count` 递减, 当打开计数器为 0 时表示该文件不再被使用, 就可从系统打开文件表中删除相应条目。

4 任务一

4.1 任务内容

实现一个简单的类 FAT-16 文件系统, 具有功能:`mkdir,ls,delete,open,read,write,close`。

4.2 任务实现规划

按照实验指导书的提示, 实现的是基于内存的简单文件系统, 不需要考虑磁盘操作 (定义数组模拟磁盘, 不需要考虑磁盘的寻道延迟、旋转延迟和传输延迟), 也不需要实现 SSD 的 `over-provisioning` 和 `garbage collection` 等。重点考虑实现 FAT、目录结构、文件结构以及相应的文件操作等即可。

4.2.1 常量定义头文件

在 `include` 目录下的 `fat.h` 文件中定义所有用到的常量、结构体如下:

常量与结构体定义

```
1 #include<stdio.h>
2 #include<string.h>
3 #include<stdlib.h>
4 #include<stdbool.h>
5
6 // 定义所用的常量
7 #define DISK_SIZE 2560 //磁盘大小
8 #define BLOCK_SIZE 64 //磁盘块大小
9 #define BLOCK_NUM (DISK_SIZE / BLOCK_SIZE)
10 #define MAX_ENTRY 32
11 #define MAX_OPENFILE 32
12 #define MAX_FD 16
13 #define DIR_NAME_LENGTH 16
```

```

14 // 定义模式
15 #define MODE_READ 1
16 #define MODE_WRITE 2
17 #define MODE_RDWR 3
18 #define MODE_WAPPEND 4 //写追加
19 // 全局数组声明
20 extern char Disk[DISK_SIZE];
21 extern int FAT[BLOCK_NUM]; // FAT表,个数即为块数。
22 extern int current_dir; //当前目录索引
23
24
25 typedef struct{
26     char name[DIR_NAME_LENGTH]; // 目录长度
27     int size;
28     size_t start_block;
29     int ac; //是否使用,0表示未使用,1表示已使用
30     int type; //0表示文件,1表示目录
31     int parent;
32 }DirEntry;
33
34 typedef struct{
35     int used; //0空闲,1已经使用
36     int dir_index; //文件下标
37     int offset; //读写偏移
38     int mode; //打开模式
39 }FileDescriptor;
40
41 extern DirEntry Directory[MAX_ENTRY];
42 extern FileDescriptor FDTTable[MAX_FD];

```

上述代码中定义了磁盘大小、磁盘块大小、磁盘块数 (磁盘大小/磁盘块大小) 以及最大目录项数、最大打开文件数、最大文件描述符数和最大文件名长度。定义了目录项结构体 DirEntry, 包含属性目录名、目录大小、起始磁盘块号、是否使用、目录类型 (0 表示文件,1 表示目录) 和父目录索引。定义了文件描述符结构体 FileDescriptor, 包含属性是否使用、文件下标、读写偏移以及打开模式 (对应只读 r、只写 w 和读写 rw)。还声明了全局变量磁盘数组 Disk 用以描述磁盘;FAT 数组表示磁盘块的链接和分配关系;current_dir 表示当前目录位置索引;Directory 数组表示目录项;FDTTable 数组表示文件描述符。

4.3 函数声明

在 include 目录下的 fat.h 中声明了实现该 FAT-16 文件系统所用到的函数, 代码如下:

函数声明

```

1 //初始化
2 void init_fs(void);
3 void init_fd_table(void);
4

```

```
5 //目录
6 bool fs_mkdir(const char *dirname);
7 bool fs_ls(void);
8 bool fs_cd(const char *dirname);
9
10
11 //文件创建与删除
12 bool fs_create(const char *filename);
13 bool fs_delete(const char *filename);
14
15 //文件操作,读写
16 int fs_open(const char * filename,int mode); //返回文件描述符fd
17 bool fs_close(int fd);
18 int fs_write(int fd,const char *data,int size);
19 int fs_read(int fd,char *buffer,int size);
20 bool fs_seek(int fd,int offset);
21
22
23 //打印提示
24 void print_prompt(void);
25
26 bool cd(const char *filename);
27 bool ls();
28 bool mkdir(const char *dirname);
```

其中主要包括了系统初始化函数、目录操作函数、文件操作函数 (目录的创建、删除和切换当前位置, 文件的创建、删除、打开、关闭、读写和定位) 以及辅助打印信息函数。

4.3.1 文件系统初始化

在 src 文件夹中的 init.c 文件中实现对该文件系统的初始化, 代码如下:

文件系统初始化

```
1 #include "fat.h"
2 #include <string.h>
3
4 // 全局变量定义
5 char Disk[DISK_SIZE];
6 int FAT[BLOCK_NUM];
7 int current_dir;
8 DirEntry Directory[MAX_ENTRY];
9 FileDescriptor FDTTable[MAX_FD];
10
11 void init_fd_table(void){
12     memset(FDTTable,0,sizeof(FDTTable));
13     for (int i = 0; i < MAX_FD;i++){
14         FDTTable[i].dir_index = -1;
15     }
```

```
16 }
17
18 void init_fs(void){
19     memset(Disk,0,sizeof(Disk));
20     memset(FAT,0,sizeof(FAT));
21     memset(Directory,0,sizeof(Directory));
22     for (int i = 0; i < MAX_ENTRY; i++){
23         Directory[i].start_block = -1;
24         Directory[i].parent = -1;
25     }
26     strcpy(Directory[0].name, "/");
27     Directory[0].size = 0;
28     Directory[0].start_block = -1;
29     Directory[0].ac = 1;
30     Directory[0].type = 1;
31     Directory[0].parent = -1;
32     current_dir = 0;
33     init_fd_table();
34 }
```

其中包括对磁盘数组、FAT 表、目录项数组和文件描述符数组的全部清零和对根目录的特殊初始化(确认已分配, 类型为目录, 父目录索引为-1), 表示当前位置的全局变量 `current_dir` 置为 0, 确保初始化后当前处于根目录。

4.3.2 文件目录操作函数

在 `src` 文件夹中的 `dir.c` 文件中实现对目录的相关操作, 包括创建文件夹、列出当前目录下所有文件及文件夹、切换目录等。具体代码如下:

目录操作函数

```
1  #include "fat.h"
2
3  //mkdir
4
5  bool mkdir(const char *dirname){
6      for (int i = 0; i < MAX_ENTRY; i++){
7          if (Directory[i].ac == 0) //找到空闲目录项
8          {
9              strncpy(Directory[i].name, dirname, DIR_NAME_LENGTH);
10             Directory[i].size = 0;
11             Directory[i].start_block = -1;
12             Directory[i].ac = 1;
13             Directory[i].type = 1; //1表示目录
14             Directory[i].parent = current_dir; //设置父目录索引
15             return true;
16         }
17     }
```

```
18     return false; // 目录创建失败返回 false
19 }
20
21 int get_dir_size(int dir_index){
22     int total = 0;
23
24     for (int i = 0; i < MAX_ENTRY; i++){
25         if (Directory[i].ac == 1 && Directory[i].parent == dir_index){
26             if (Directory[i].type == 0){
27                 //普通文件
28                 total += Directory[i].size;
29             }else if (Directory[i].type == 1){
30                 //目录文件,递归统计
31                 total += get_dir_size(i);
32             }
33         }
34     }
35     return total; //统计 dir_index 号目录下的文件、目录项总个数
36 }
37
38 //ls
39 bool ls(){
40     for (int i = 0; i < MAX_ENTRY; i++){
41         //遍历所有 Directory, 已分配且在当前目录下的文件和目录 list 出来
42         if (Directory[i].ac == 1 && Directory[i].parent == current_dir){
43             int show_size;
44             if (Directory[i].type == 1){
45                 show_size = get_dir_size(i);
46             }else{
47                 show_size = Directory[i].size;
48             }
49
50             printf("%s\t%s\t%d bytes\n",
51                 Directory[i].type == 1 ? "DIR":"FILE",
52                 Directory[i].name,
53                 show_size
54             );
55         }
56     }
57     return true;
58 }
59
60 // cd
61 bool cd(const char *dirname){
62     if (dirname == NULL) return false;
63
64     // 回到根目录
```



```

65     if (strcmp(dirname, "/") == 0){
66         current_dir = 0;
67         return true;
68     }
69
70     // 回到父目录
71     if (strcmp(dirname, "..") == 0){
72         if (Directory[current_dir].parent != -1){ // 根目录往上翻还是在根目录
73             current_dir = Directory[current_dir].parent;
74         }
75         return true;
76     }
77
78
79     // 进入下级目录
80     for (int i = 0; i < MAX_ENTRY; i++){
81         if (Directory[i].ac == 1 && Directory[i].parent == current_dir &&
82             Directory[i].type == 1 &&
83             strcmp(Directory[i].name, dirname) == 0){
84                 current_dir = i;
85                 return true;
86             }
87     }
88     return false;
89 }

```

其中新建文件夹就是在 Directory 数组中找一个空闲目录项, 将其设置为指定的新建目录项并更新对应的父目录索引、大小、类型等。辅助函数 get_dir_size() 采用递归的方式, 统计指定目录下所有文件及子目录的总大小。ls() 函数用于列出当前目录下所有文件及文件夹, 打印出对应的类型、文件名和大小, 实现方法为遍历 Directory 数组, 已分配且父目录索引等于当前目录项索引的即为当前目录下的文件、文件夹。cd() 函数用于切换当前目录, 事实上只需要改变存放当前目录位置的全局变量即可实现切换。为了简化, 只实现了切换到根目录 ('/')、父目录 ('..') 和切换到下级指定文件夹的功能。另外为了鲁棒性, 当处于根目录时, 切换到根目录操作会被跳过, 不会重复改变当前目录。

4.3.3 文件的打开与关闭

在 src 文件夹中的 fd.c 文件中实现对文件的打开与关闭, 这里打开与关闭文件是便于后续对文件的读写操作, 具体代码如下:

文件的打开与关闭

```

1  #include "fat.h"
2
3  int fs_open(const char *filename, int mode){
4      if (filename == NULL) return -1;
5
6      // 在当前目录下找普通文件
7      int dir_idx = -1;

```

```
8     for(int i = 0; i < MAX_ENTRY; i++){
9         if(Directory[i].ac == 1 &&
10            Directory[i].parent == current_dir &&
11            Directory[i].type == 0&&
12            strcmp(Directory[i].name,filename) == 0
13        ){
14            dir_idx = i;
15            break;
16        }
17    }
18    if (dir_idx == -1){
19        //没有找到
20        printf("open failed : file not found : %s \n",filename);
21        return -1;
22    }
23
24    // 找空闲fd
25    for (int fd = 0; fd < MAX_FD; fd ++){
26        if(FDTable[fd].used == 0){
27            FDTable[fd].used = 1;
28            FDTable[fd].dir_index = dir_idx;
29            FDTable[fd].offset = 0;
30            FDTable[fd].mode = mode;
31            printf("open success : %s,fd = %d\n",filename,fd);
32            return fd;
33        }
34    }
35    printf("open failed:too many open files\n");
36    return -1;
37 }
38
39 bool fs_close(int fd){
40     //无效fd
41     if (fd < 0 || fd > MAX_FD){
42         printf("close failed : invalid fd\n");
43         return false;
44     }
45     //文件还未打开
46     if(FDTable[fd].used == 0){
47         printf("close failed : fd is not open yet\n");
48         return false;
49     }
50     //正常关闭文件
51     FDTable[fd].used = 0;
52     FDTable[fd].dir_index = -1;
53     FDTable[fd].offset = 0;
54     FDTable[fd].mode = 0;
```

```

55     printf("close success: fd = %d\n", fd);
56     return true;
57 }
58
59 bool fs_seek(int fd, int offset){
60     if(fd < 0 || fd >= MAX_FD) return false;
61     if(FDTable[fd].used == 0) return false;
62     if(offset < 0) return false;
63
64     int dir_idx = FDTable[fd].dir_index;
65     int file_size = Directory[dir_idx].size;
66
67     // 读模式不允许超过文件大小
68     if(FDTable[fd].mode == MODE_READ || FDTable[fd].mode == MODE_RDWR){
69         if(offset > file_size){
70             printf("seek failed: offset %d exceeds file size %d\n", offset,
71                 file_size);
72             return false;
73         }
74     }
75
76     FDTable[fd].offset = offset;
77     return true;
78 }

```

其中 `fs_open()` 函数用于打开当前目录下指定文件名的文件 (这里为了简化实现, 没有实现打开任意目录下的文件), 通过遍历 `Directory` 数组, 找到不为空、文件名为指定文件且父目录为当前目录的文件即为要打开的文件, 找不到就返回失败并打印错误信息。找到后再在 `FDTable` 数组 (即打开文件表) 中查找未使用的项, 将其信息同步为当前要打开的文件, 并将 `FDTable` 数组中对应项的索引返回 (即文件描述符), 后续对文件的读写操作都基于该索引。 `fs_close()` 函数用于关闭指定文件, 因为确定只有确定已经打开的文件才需要关闭, 因此这里关闭文件函数接收的参数为文件描述符 `fd`。具体实现为检查 `fd` 未越界且 `FDTable` 数组中对应项不为空时, 将这一项清零即可。 `fs_seek()` 函数用于设置文件从何处开始读写, 通过接收参数文件描述符 `fd` 和偏移量 `offset`, 在确认都没有越界的情况下更新 `FDTable` 中对应项的 `offset` 属性即可。

4.4 文件读写与删除

在 `src` 文件夹中的 `file.c` 文件中实现文件的读写与删除功能, 具体包括文件的创建、读写和删除操作, 具体代码分别如下:

- 文件创建函数:

文件创建

```

1 // 创建文件, 写文件
2 bool fs_create(const char*filename){
3     for (int i = 0; i < MAX_ENTRY; i++){
4         if(Directory[i].ac == 0){

```

```
5 // 找到空闲块
6 strncpy(Directory[i].name,filename,DIR_NAME_LENGTH);
7 Directory[i].size = 0;
8 Directory[i].start_block = -1;
9 Directory[i].ac = 1;
10 Directory[i].type = 0; //0表示文件
11 Directory[i].parent = current_dir; //设置父目录索引
12 printf("File created successfully : %s\n",filename);
13 return true;
14 }
15 }
16 printf("Failed to created file : %s\n",filename);
17 return false;
18 }
19 }
```

文件创建就是接收参数文件名 filename, 在 Directory 数组中找一个空闲块, 并设置该块对应属性, 包括文件名、文件大小、文件类型、是否被分配以及父目录索引。

- 写文件函数:

写文件函数

```
1 int fs_write(int fd,const char* data,int size){
2     if(fd < 0 || fd >= MAX_FD || data == NULL || size < 0) return -1;
3     if(FDTable[fd].used == 0){
4         printf("write failed : fd is not open\n");
5         return -1;
6     }
7
8     if(FDTable[fd].mode != MODE_WRITE && FDTable[fd].mode != MODE_RDWR &&
9        FDTable[fd].mode != MODE_WAPPEND){
10        printf("write failed : fd is not writable");
11        return -1;
12    }
13
14    if (size == 0){
15        return 0;
16    }
17
18    int dir_index = FDTable[fd].dir_index;
19    int offset = FDTable[fd].offset;
20    int end_pos = offset + size;
21
22    int need_blocks = (end_pos + BLOCK_SIZE - 1) / BLOCK_SIZE;
23
24    // 确保文件有足够多的 block
25    if (Directory[dir_index].start_block == -1) {
26        int new_block = -1;
```

```
26
27     for (int i = 0; i < BLOCK_NUM; i++) {
28         if (FAT[i] == 0) {
29             new_block = i;
30             break;
31         }
32     }
33
34     if (new_block == -1) {
35         printf("write failed: no free block\n");
36         return -1;
37     }
38
39     FAT[new_block] = -1;
40     memset(Disk + new_block * BLOCK_SIZE, 0, BLOCK_SIZE);
41     Directory[dir_index].start_block = new_block;
42 }
43
44 int cur = Directory[dir_index].start_block;
45 int block_count = 1;
46
47 while (FAT[cur] != -1) {
48     cur = FAT[cur];
49     block_count++;
50 }
51
52 while (block_count < need_blocks) {
53     int new_block = -1;
54
55     for (int i = 0; i < BLOCK_NUM; i++) {
56         if (FAT[i] == 0) {
57             new_block = i;
58             break;
59         }
60     }
61
62     if (new_block == -1) {
63         printf("write failed: no enough space\n");
64         return -1;
65     }
66
67     FAT[new_block] = -1;
68     memset(Disk + new_block * BLOCK_SIZE, 0, BLOCK_SIZE);
69
70     FAT[cur] = new_block;
71     cur = new_block;
72     block_count++;
```

```
73     }
74
75     //找offset对应的block
76     int block_index = offset / BLOCK_SIZE;
77     int block_offset = offset % BLOCK_SIZE;
78
79     cur = Directory[dir_index].start_block;
80
81     for(int i = 0; i < block_index; i++){
82         cur = FAT[cur];
83     }
84
85     //从offset开始写数据
86     int bytes_written = 0;
87     while (cur != -1 && bytes_written < size){
88         int write_size = BLOCK_SIZE - block_offset;
89
90         if(write_size > size - bytes_written){
91             write_size = size - bytes_written;
92         }
93         memcpy(
94             Disk + cur * BLOCK_SIZE + block_offset,
95             data + bytes_written,
96             write_size
97         );
98         bytes_written += write_size;
99         block_offset = 0;
100         if(bytes_written < size){
101             cur = FAT[cur];
102         }
103     }
104
105     //更新offset和文件大小
106
107     FDTTable[fd].offset += bytes_written;
108     if(FDTTable[fd].offset > Directory[dir_index].size){
109         Directory[dir_index].size = FDTTable[fd].offset;
110     }
111
112     return bytes_written;
113 }
114
```

写文件函数接收参数文件描述符 `fd`, 数据指针 `data` 以及数据大小 `size`, 实际写入前需要确保 `fd` 有效且文件可写, 同时确保 `FAT` 中有足够的多的块写入内容 (否则写文件失败), 最后根据 `offset` 确认写入的块的位置并写入 `data`。

- 读文件函数:

读文件函数

```
1 int fs_read(int fd, char *buffer, int size){
2
3     if (fd < 0 || fd >= MAX_FD || buffer == NULL || size < 0) {
4         return -1;
5     }
6
7     if (FDTTable[fd].used == 0) {
8         printf("read failed: file is not open\n");
9         return -1;
10    }
11
12    if (FDTTable[fd].mode != MODE_READ && FDTTable[fd].mode != MODE_RDWR ){
13        printf("read failed : file is not readable\n");
14        return -1;
15    }
16
17    int dir_index = FDTTable[fd].dir_index;
18    int file_size = Directory[dir_index].size;
19    int offset = FDTTable[fd].offset;
20
21    if (offset >= file_size) {
22        buffer[0] = '\0';
23        return 0;
24    }
25
26    int bytes_to_read = size;
27    if (offset + bytes_to_read > file_size) {
28        bytes_to_read = file_size - offset;
29    }
30
31    int bytes_read = 0;
32    int cur = Directory[dir_index].start_block;
33    int skip_blocks = offset / BLOCK_SIZE;
34    int block_offset = offset % BLOCK_SIZE;
35
36    // 跳到 offset 所在的块
37    for (int i = 0; i < skip_blocks && cur != -1; i++) {
38        cur = FAT[cur];
39    }
40
41    while (cur != -1 && bytes_read < bytes_to_read) {
42        int read_size = BLOCK_SIZE - block_offset;
43
44        if (read_size > bytes_to_read - bytes_read) {
45            read_size = bytes_to_read - bytes_read;
```

```
46
47     memcpy(
48         buffer + bytes_read,
49         Disk + cur * BLOCK_SIZE + block_offset,
50         read_size
51     );
52
53     bytes_read += read_size;
54     block_offset = 0;
55     cur = FAT[cur];
56 }
57
58 buffer[bytes_read] = '\0';
59 FDTTable[fid].offset += bytes_read;
60
61 return bytes_read;
62 }
63
```

读文件函数接收参数文件描述符 fd, 数据指针 buffer 以及要读的数据大小 size, 类似写文件函数, 同样要确保 fd 有效且文件可读, 同时根据 offset 确认读取的块的位置并读取数据。

- 删除文件函数:

删除文件函数

```
1 bool fs_delete(const char *filename){
2     for (int i = 0 ; i < MAX_ENTRY; i ++){
3         if(Directory[i].ac == 1 && Directory[i].parent == current_dir &&
4            Directory[i].type == 0 && strcmp(Directory[i].name,filename) == 0){
5             //文件占用的磁盘块在FAT中全部标记为0
6             int cur = Directory[i].start_block;
7             while (cur != -1){
8                 int next = FAT[cur];
9                 FAT[cur] = 0;
10                if(next == -1){//找到末尾块
11                    break;
12                }
13                cur = next;
14            }
15            Directory[i].ac = 0;
16            printf("File deleted successfully : %s \n",filename);
17            return true;
18        }
19    }
20    printf("Failed to delete file: %s\n", filename);
21    return false;
22 }
```


删除文件函数类似文件创建函数, 接收参数文件名 filename, 然后在 Directory 数组中查找该文件, 找到后释放该文件占用的所有 FAT 块 (从 start_block 开始全部置 0), 并设置为未分配状态即可。

4.4.1 文件系统入口

在 src 文件夹中的 main 函数中实现整个文件系统的入口, 实现思路为先初始化文件系统, 然后在死循环中等待用户输入命令, 利用之前实现的基础的文件和目录操作函数来实现复杂用户命令, 具体代码如下:

```
1
2 void print_path_recursive(int dir){
3     if(dir == 0){
4         printf("/");
5         return;
6     }
7
8     int parent = Directory[dir].parent;
9     print_path_recursive(parent);
10
11     if (parent != 0){
12         printf("/");
13     }
14
15     printf("%s",Directory[dir].name);
16 }
17
18 void print_prompt(){
19     printf("SimpleFAT:");
20     print_path_recursive(current_dir);
21     printf("$ ");
22 }
23
24 int main(){
25     char line[256];
26     char cmd[32];
27     char arg1[64];
28     char arg2[128];
29     char buffer[1024];
30
31     init_fs();
32
33     while(1){
34         print_prompt();
```

```
35     if(fgets(line,sizeof(line),stdin) == NULL){
36         break;
37     }
38     //fgets会读入\n
39     line[strcspn(line,"\n")] = '\0';
40     if (strlen(line) == 0){
41         continue;
42     }
43     cmd[0] = '\0';
44     arg1[0] = '\0';
45     arg2[0] = '\0';
46
47     sscanf(line, "%31s %63s %127[^\n]", cmd, arg1, arg2);
48     if (strcmp(cmd, "exit") == 0) {
49         break;
50     }
51
52     else if (strcmp(cmd, "ls") == 0) {
53         ls();
54     }
55
56     else if (strcmp(cmd, "mkdir") == 0) {
57         if (strlen(arg1) == 0) {
58             printf("usage: mkdir dirname\n");
59         } else if (!mkdir(arg1)) {
60             printf("mkdir failed: %s\n", arg1);
61         }
62     }
63
64     else if (strcmp(cmd, "cd") == 0) {
65         if (strlen(arg1) == 0) {
66             printf("usage: cd dirname\n");
67         } else if (!cd(arg1)) {
68             printf("cd failed: %s\n", arg1);
69         }
70     }
71
72     else if (strcmp(cmd, "touch") == 0) {
73         if (strlen(arg1) == 0) {
74             printf("usage: touch filename\n");
75         } else {
76             fs_create(arg1);
77         }
78     }
79
80     else if (strcmp(cmd, "rm") == 0) {
81         if (strlen(arg1) == 0) {
```

```
82         printf("usage: rm filename\n");
83     } else {
84         fs_delete(arg1);
85     }
86 }
87
88 else if (strcmp(cmd, "open") == 0) {
89     if (strlen(arg1) == 0 || strlen(arg2) == 0) {
90         printf("usage: open filename mode\n");
91         printf("mode: r | w | rw\n");
92         continue;
93     }
94
95     int mode;
96
97     if (strcmp(arg2, "r") == 0) {
98         mode = MODE_READ;
99     } else if (strcmp(arg2, "w") == 0) {
100         mode = MODE_WRITE;
101     } else if (strcmp(arg2, "rw") == 0) {
102         mode = MODE_RDWR;
103     } else {
104         printf("invalid mode: %s\n", arg2);
105         continue;
106     }
107
108     int fd = fs_open(arg1, mode);
109     if (fd >= 0) {
110         printf("opened fd = %d\n", fd);
111     }
112 }
113
114 else if (strcmp(cmd, "close") == 0) {
115     if (strlen(arg1) == 0) {
116         printf("usage: close fd\n");
117     } else {
118         int fd = atoi(arg1);
119         fs_close(fd);
120     }
121 }
122
123 else if (strcmp(cmd, "write") == 0) {
124     if (strlen(arg1) == 0 || strlen(arg2) == 0) {
125         printf("usage: write fd content\n");
126     } else {
127         int fd = atoi(arg1);
128         fs_write(fd, arg2, strlen(arg2));
```

```
129     }
130 }
131
132 else if (strcmp(cmd, "read") == 0) {
133     if (strlen(arg1) == 0) {
134         printf("usage: read fd [size]\n");
135         printf("    if size omitted, reads entire file from current offset\n");
136     } else {
137         int fd = atoi(arg1);
138
139         if (fd < 0 || fd >= MAX_FD || FdTable[fd].used == 0) {
140             printf("read failed: invalid fd %d\n", fd);
141             continue;
142         }
143
144         int size;
145         if (strlen(arg2) == 0) {
146             // 未指定 size, 读取文件剩余全部内容
147             int dir_idx = FdTable[fd].dir_index;
148             int file_size = Directory[dir_idx].size;
149             int offset = FdTable[fd].offset;
150             size = file_size - offset;
151             if (size <= 0 || size > (int)sizeof(buffer) - 1) {
152                 size = (int)sizeof(buffer) - 1;
153             }
154         } else {
155             size = atoi(arg2);
156         }
157
158         int n = fs_read(fd, buffer, size);
159
160         if (n >= 0) {
161             buffer[n] = '\0';
162             printf("%s\n", buffer);
163         }
164     }
165 }
166
167 else if (strcmp(cmd, "cat") == 0) {
168     if (strlen(arg1) == 0) {
169         printf("usage: cat filename\n");
170     } else {
171         int fd = fs_open(arg1, MODE_READ);
172         if (fd >= 0) {
173             char cat_buf[1024];
174             int n = fs_read(fd, cat_buf, sizeof(cat_buf) - 1);
```

```
175         if (n >= 0) {
176             cat_buf[n] = '\0';
177             printf("%s\n", cat_buf);
178         }
179         fs_close(fd);
180     }
181 }
182 }
183
184 else if (strcmp(cmd, "seek") == 0) {
185     if (strlen(arg1) == 0 || strlen(arg2) == 0) {
186         printf("usage: seek fd offset\n");
187     } else {
188         int fd = atoi(arg1);
189         int offset = atoi(arg2);
190
191         if (!fs_seek(fd, offset)) {
192             printf("seek failed\n");
193         }
194     }
195 }
196
197 else if (strcmp(cmd, "cls") == 0) {
198     printf("\033[2J\033[H");
199     fflush(stdout);
200 }
201
202 else if (strcmp(cmd, "help") == 0) {
203     printf("commands:\n");
204     printf("  ls\n");
205     printf("  mkdir dirname\n");
206     printf("  cd dirname\n");
207     printf("  touch filename\n");
208     printf("  rm filename\n");
209     printf("  cat filename\n");
210     printf("  open filename r|w|rw\n");
211     printf("  close fd\n");
212     printf("  write fd content\n");
213     printf("  read fd [size]\n");
214     printf("  seek fd offset\n");
215     printf("  cls\n");
216     printf("  exit\n");
217 }
218
219 else {
220     printf("unknown command: %s\n", cmd);
221     printf("type 'help' for help\n");
```

```

222     }
223 }
224 return 0;
225 }

```

其中 `print_path_recursive()` 函数采用递归的方式打印当前路径;`print_prompt()` 函数用于实现类似打印当前路径提示符;`main()` 函数中通过接收用户输入命令参数, 在 `if-else` 分支中根据不同命令参数调用不同函数实现文件系统功能, 如 `cat` 命令实现读一个文件内容并打印出来, 采用的就是 `open+read+close` 的组合实现。(这里没有实现 Linux 中完整的 `cat` 命令所有功能, 只是实现了查看指定文件名文件功能)。

4.4.2 FAT-16 文件系统使用测试

将上述实现的 FAT-16 文件系统编译后运行, 测试结果如下:

```

[root@LYJ FAT-16]# make
mkdir -p obj
gcc -Iinclude -Wall -g -c src/dir.c -o obj/dir.o
gcc -Iinclude -Wall -g -c src/fd.c -o obj/fd.o
gcc -Iinclude -Wall -g -c src/file.c -o obj/file.o
gcc -Iinclude -Wall -g -c src/init.c -o obj/init.o
gcc -Iinclude -Wall -g -c src/main.c -o obj/main.o
mkdir -p bin
gcc obj/dir.o obj/fd.o obj/file.o obj/init.o obj/main.o -o bin/app
[root@LYJ FAT-16]# ./bin/app
SimpleFAT:/$ mkdir hello
SimpleFAT:/$ ls
DIR    hello    0 bytes
SimpleFAT:/$ cd hello
SimpleFAT:/hello$ cd ..
SimpleFAT:/$

```

图 1: 创建文件夹和切换目录

```

SimpleFAT:/$ touch hello.txt
File created successfully : hello.txt
SimpleFAT:/$ open hello.txt w
open success : hello.txt,fd = 0
opened fd = 0
SimpleFAT:/$ write 0 hello,world!
SimpleFAT:/$ close 0
close success: fd = 0
SimpleFAT:/$ ls
DIR    hello    0 bytes
FILE    hello.txt    12 bytes
SimpleFAT:/$ open hello.txt r
open success : hello.txt,fd = 0
opened fd = 0
SimpleFAT:/$ read 0
hello,world!
SimpleFAT:/$ close 0
close success: fd = 0
SimpleFAT:/$ cat hello.txt
open success : hello.txt,fd = 0
hello,world!
close success: fd = 0
SimpleFAT:/$

```

图 2: 创建文件和读写文件

```
SimpleFAT:/$ ls
DIR      hello      0 bytes
FILE     hello.txt   12 bytes
SimpleFAT:/$ rm hello.txt
File deleted successfully : hello.txt
SimpleFAT:/$ ls
DIR      hello      0 bytes
SimpleFAT:/$
```

图 3: 删除文件

从上述测试结果截图可以看到, 该 FAT-16 文件系统能成功实现基于内存的单进程文件系统, 正确完成了 mkdir、ls、cd、touch、cat、open、read、write、close、rm 等功能, 任务一成功完成。

4.5 任务一函数汇总

下表对任务一实现的所有主要函数进行汇总, 以便对照查阅。

表 1: 任务一主要函数汇总

函数名	功能	实现方法	关键数据结构
init_fs()	初始化文件系统	清零 Disk/FAT/Directory 数组, 初始化根目录项	Disk[], FAT[], Directory[]
init_fd_table()	初始化文件描述符表	memset 清零, dir_index 置为-1	FDTable[]
mkdir()	创建目录	遍历 Directory 找空闲项, 设置目录名/类型/父目录索引	Directory[]
ls()	列出当前目录内容	遍历 Directory, 匹配 parent==current_dir, 打印类型/名称/大小	Directory[], current_dir
cd()	切换当前目录	修改 current_dir, 支持/、..、下级子目录名	Directory[], current_dir
fs_create()	创建文件	在 Directory 中找空闲项, 设置文件名/类型/父目录索引	Directory[]
fs_open()	打开文件	遍历 Directory 查找文件 → 在 FDTable 中分配空闲表项 → 返回 fd	Directory[], FDTable[]
fs_close()	关闭文件	验证 fd 有效性 → 将 FDTable 对应表项清零	FDTable[]
fs_read()	读取文件内容	从 offset 所在块开始, 沿 FAT 链逐块 memcpy 到 buffer	Disk[], FAT[], FDTable[]
fs_write()	写入文件内容	确保 FAT 链有足够块 → 从 offset 逐块写入 data → 更新文件大小	Disk[], FAT[], FDTable[]
fs_delete()	删除文件	沿 FAT 链释放所有块 (FAT 标记为 0), 目录项标记为未使用	Directory[], FAT[]
fs_seek()	设置读写偏移	验证 fd 和 offset 有效性 → 更新 FDTable 中 offset 字段	FDTable[]

5 任务二

5.1 任务内容

实现多进程同时访问文件、目录功能。

5.2 任务实现规划

该问题即是实现读者写者问题, 其中读者进程可以同时访问文件和目录, 写者进程必须与读者进程和其他写者进程互斥地访问文件 (即写者创建、删除文件和目录必须是互斥的)。由于真正的 fork() 需要共享内存, 信号量需要在父子进程间通信, 实现起来过于复杂, 因此采用 pthread 线程模拟进程 (线程可以自然共享全局变量)。实现进程同步时采用经典读者写者问题的解决方法, 其中需要互斥的操作如创建和删除文件、目录; 写文件和读文件等需要信号量控制互斥地打开, 因此涉及这些操作的函数必须加锁。

5.2.1 头文件常量定义与函数声明

在 include 文件夹中的 fat.h 文件中需要修改原来的 DirEntry 结构体, 加上属于该文件的读写锁 rwlock; 定义全局互斥锁用于保护修改读者数量的信号量 fs_mutex; 定义进程控制块结构体 PCB, 用以维护当前进程信息。具体代码如下:

fat.h

```
1  /*系统是基于内存的: 建立一个数组,把这个数组当成硬盘,实现文件系统。
2  假设只有一个进程使用。*/
3  #include <stdio.h>
4  #include <string.h>
5  #include <stdlib.h>
6  #include <stdbool.h>
7  #include <pthread.h>
8  #include <semaphore.h>
9
10 //定义所用常量
11 #define DISK_MAXLEN 2560
12 #define BLOCK_SIZE 64
13 #define BLOCK_NUM (DISK_MAXLEN / BLOCK_SIZE)
14 #define MAX_ENTRY 32
15 #define MAX_OPEN_FILE 32
16 #define MAX_FD 16
17
18 //最大进程数
19 #define MAX_PROCESS 8
20
21 //模式定义
22 #define MODE_READ 1
23 #define MODE_WRITE 2
24 #define MODE_RDWR 3
25 #define MODE_WAPPEND 4
26
27 //目录项
28 typedef struct {
29     char name[16];
30     int size;
31     int start_block;
32     int ac;
33     int type;
34     int parent;
35     int reader_count; /* 当前读者数量 */
36     sem_t rw_lock; /* 文件资源信号量,初值=1 */
37     pthread_mutex_t mutex; /* 保护 reader_count 的互斥锁 */
38 } DirEntry;
39
40 //文件描述符
41 typedef struct {
```



```

42     int used;           // 0 空闲,1 已使用
43     int dir_index;      // 对应 Directory[] 下标
44     int offset;         // 当前读写偏移
45     int mode;           // 打开模式
46 } FileDescriptor;
47
48 //pcb
49 typedef struct {
50     int pid;
51     FileDescriptor fd_table[MAX_FD];
52     int current_dir;      // 每个进程维护的当前目录索引
53 } PCB;
54
55
56 extern char Disk[DISK_MAXLEN];
57 extern int FAT[BLOCK_NUM];
58 extern DirEntry Directory[MAX_ENTRY];
59 extern pthread_mutex_t fs_mutex;
60 extern PCB ProcessTable[MAX_PROCESS];
61
62
63 void init_fs(void);
64 void init_process_table(void);
65
66 //目录操作
67 bool mkdir(int pid, const char *dirname);
68 bool ls(int pid);
69 bool cd(int pid, const char *dirname);
70
71 //文件创建与删除
72 bool fs_create(int pid, const char *filename);
73 bool fs_delete(const char *filename);
74
75 //文件打开关闭与读写
76 int fs_open(int pid, const char *filename, int mode);
77 bool fs_close(int pid, int fd);
78 int fs_read(int pid, int fd, char *buffer, int size);
79 int fs_write(int pid, int fd, const char *data, int size);
80 bool fs_seek(int pid, int fd, int offset);
81
82
83 void print_prompt(int pid);

```

其中相较于任务一的主要修改为: 改为每个进程单独维护当前目录索引 `current_dir`, 而不是全局维护一个; 所有目录、文件操作函数都需要添加 `pid` 参数用以指定当前进程; 目录项结构体 `DirEntry` 中新增 `reader_count` 字段用以维护当前读者数量, `rw_lock` 信号量用于保护文件资源, `mutex` 互斥锁用来保护对 `reader_count` 的修改。

5.2.2 系统初始化

在 src 文件夹下的 init.c 文件中实现文件系统初始化, 其中需要额外实现 PCB 数组的初始化, 代码如下:

系统初始化

```
1  #include "fat.h"
2  #include <string.h>
3
4  char Disk[DISK_MAXLEN];
5  int FAT[BLOCK_NUM];
6  DirEntry Directory[MAX_ENTRY];
7  pthread_mutex_t fs_mutex = PTHREAD_MUTEX_INITIALIZER;
8  PCB ProcessTable[MAX_PROCESS];
9
10 void init_process_table(void)
11 {
12     memset(ProcessTable, 0, sizeof(ProcessTable));
13
14     for (int p = 0; p < MAX_PROCESS; p++) {
15         ProcessTable[p].pid = p;
16         ProcessTable[p].current_dir = 0;    // 每个进程初始在根目录
17
18         for (int fd = 0; fd < MAX_FD; fd++) {
19             ProcessTable[p].fd_table[fd].dir_index = -1;
20             ProcessTable[p].fd_table[fd].offset = 0;
21             ProcessTable[p].fd_table[fd].mode = 0;
22             ProcessTable[p].fd_table[fd].used = 0;
23         }
24     }
25 }
26 void init_fs(void)
27 {
28     memset(Disk, 0, sizeof(Disk));
29     memset(FAT, 0, sizeof(FAT));
30     memset(Directory, 0, sizeof(Directory));
31
32     for (int i = 0; i < MAX_ENTRY; i++) {
33         Directory[i].start_block = -1;
34         Directory[i].parent = -1;
35         Directory[i].reader_count = 0;
36         sem_init(&Directory[i].rw_lock, 0, 1);
37         pthread_mutex_init(&Directory[i].mutex, NULL);
38     }
39
40     strcpy(Directory[0].name, "/");
41     Directory[0].size = 0;
42     Directory[0].start_block = -1;
```

```
43     Directory[0].ac = 1;
44     Directory[0].type = 1;
45     Directory[0].parent = -1;
46
47     init_process_table();
48 }
```

这里类似任务一的初始化, 只是新增了对 PCB 数组的初始化以及在 DirEntry 结构体数组中信号量的初始化。

5.2.3 文件目录操作

在 src 文件夹中的 dir.c 文件中实现多进程下访问目录功能, 这里显然创建目录、列出当前目录下所有文件及文件夹操作都需要互斥进行, 具体代码如下:

目录操作函数

```
1  #include "fat.h"
2
3  bool mkdir(int pid, const char *dirname)
4  {
5      if (pid < 0 || pid >= MAX_PROCESS) return false;
6      int cur_dir = ProcessTable[pid].current_dir;
7
8      pthread_mutex_lock(&fs_mutex); //创建文件需要加锁
9
10     for (int i = 0; i < MAX_ENTRY; i++) {
11         if (Directory[i].ac == 0) {
12             strncpy(Directory[i].name, dirname, 16);
13             Directory[i].size = 0;
14             Directory[i].start_block = -1;
15             Directory[i].ac = 1;
16             Directory[i].type = 1;
17             Directory[i].parent = cur_dir;
18             Directory[i].reader_count = 0;
19             sem_init(&Directory[i].rw_lock, 0, 1);
20             pthread_mutex_init(&Directory[i].mutex, NULL);
21             pthread_mutex_unlock(&fs_mutex);
22             return true;
23         }
24     }
25
26     pthread_mutex_unlock(&fs_mutex);
27     return false;
28 }
29
30 static int get_dir_size(int dir_index)
31 {
```

```
32     int total = 0;
33     for (int i = 0; i < MAX_ENTRY; i++) {
34         if (Directory[i].ac == 1 && Directory[i].parent == dir_index) {
35             if (Directory[i].type == 0) {
36                 total += Directory[i].size;
37             } else if (Directory[i].type == 1) {
38                 total += get_dir_size(i);
39             }
40         }
41     }
42     return total;
43 }
44
45 bool ls(int pid)
46 {
47     if (pid < 0 || pid >= MAX_PROCESS) return false;
48     int cur_dir = ProcessTable[pid].current_dir;
49
50     pthread_mutex_lock(&fs_mutex);
51
52     for (int i = 0; i < MAX_ENTRY; i++) {
53         if (Directory[i].ac == 1 && Directory[i].parent == cur_dir) {
54             int show_size;
55             if (Directory[i].type == 1) {
56                 show_size = get_dir_size(i);
57             } else {
58                 show_size = Directory[i].size;
59             }
60             printf("%s\t%s\t%d bytes\n",
61                 Directory[i].type == 1 ? "DIR" : "FILE",
62                 Directory[i].name,
63                 show_size);
64         }
65     }
66
67     pthread_mutex_unlock(&fs_mutex);
68     return true;
69 }
70
71 bool cd(int pid, const char *dirname)
72 {
73     if (pid < 0 || pid >= MAX_PROCESS || dirname == NULL) return false;
74
75     int cur_dir = ProcessTable[pid].current_dir;
76
77     if (strcmp(dirname, "/") == 0) {
78         ProcessTable[pid].current_dir = 0;
```

```

79     return true;
80 }
81
82 if (strcmp(dirname, "..") == 0) {
83     if (Directory[cur_dir].parent != -1) {
84         ProcessTable[pid].current_dir = Directory[cur_dir].parent;
85     }
86     return true;
87 }
88
89 //遍历以在当前目录下找子目录(对Directory[]只读),避免其他进程此时mkdir/touch,
90 //加锁保证一致性
91 pthread_mutex_lock(&fs_mutex);
92
93 for (int i = 0; i < MAX_ENTRY; i++) {
94     if (Directory[i].ac == 1 &&
95         Directory[i].parent == cur_dir &&
96         Directory[i].type == 1 &&
97         strcmp(Directory[i].name, dirname) == 0) {
98         ProcessTable[pid].current_dir = i;
99         pthread_mutex_unlock(&fs_mutex);
100         return true;
101     }
102 }
103 pthread_mutex_unlock(&fs_mutex);
104 return false;
105 }

```

目录操作有关函数首先需要提取进程控制块中存放的当前目录位置作为实际工作目录。mkdir() 函数在查找空闲目录项前需要申请信号量 (pthread_mutex_lock(&fs_mutex)), 查找完成后成功获取空闲 Directory 项作为新建目录项或查找失败, 都需要释放信号量 (pthread_mutex_unlock(&fs_mutex)), 其中成功获取空闲项并修改的操作与任务一相同。类似地, ls() 函数和 cd() 函数的核心操作与任务一中类似, 只是在需要互斥访问目录项数组 Directory 时需要前后加信号量保护。

5.2.4 文件打开与关闭

在 src 文件夹的 file.c 文件中实现对文件的打开与关闭操作, 由于这里每个进程实际都有可能是写者和读者, 因此打开文件时不区分是读进程还是写进程, 只根据打开文件的模式来判断, 具体代码如下:

- 打开文件

打开文件函数

```

1 int fs_open(int pid, const char *filename, int mode)
2 {
3     if (pid < 0 || pid >= MAX_PROCESS || filename == NULL) {

```

```
4         return -1;
5     }
6
7     int cur_dir = ProcessTable[pid].current_dir;
8     int dir_index = -1;
9
10    //互斥访问Directory,查找要打开的文件
11    pthread_mutex_lock(&fs_mutex);
12
13    for (int i = 0; i < MAX_ENTRY; i++) {
14        if (Directory[i].ac == 1 &&
15            Directory[i].parent == cur_dir &&
16            Directory[i].type == 0 &&
17            strcmp(Directory[i].name, filename) == 0) {
18            dir_index = i;
19            break;
20        }
21    }
22
23    if (dir_index == -1) {
24        pthread_mutex_unlock(&fs_mutex);
25        printf("open failed: file not found: %s\n", filename);
26        return -1;
27    }
28
29    pthread_mutex_unlock(&fs_mutex);
30
31    //只读、读写打开文件需要修改reader_count数量(互斥修改)
32
33    if (mode == MODE_READ) {
34        pthread_mutex_lock(&Directory[dir_index].mutex); //互斥修改
35        reader_count
36        Directory[dir_index].reader_count++;
37        if (Directory[dir_index].reader_count == 1) {
38            if (sem_trywait(&Directory[dir_index].rw_lock) != 0) { //有写者
39                //时打开失败
40                Directory[dir_index].reader_count--;
41                pthread_mutex_unlock(&Directory[dir_index].mutex);
42                printf("open failed: file is busy\n");
43                return -1;
44            }
45        }
46        pthread_mutex_unlock(&Directory[dir_index].mutex); //修改
47        reader_count结束
48    }
49
50    else if (mode == MODE_WRITE || mode == MODE_RDWR) {
51        if (sem_trywait(&Directory[dir_index].rw_lock) != 0) { //文件被读
```

```

48         printf("open failed: file is busy\n");
49         return -1;
50     }
51 }
52 else {
53     printf("open failed: invalid mode\n");
54     return -1;
55 }
56
57 //分配fd
58 for (int fd = 0; fd < MAX_FD; fd++) {
59     if (ProcessTable[pid].fd_table[fd].used == 0) {
60         ProcessTable[pid].fd_table[fd].used = 1;
61         ProcessTable[pid].fd_table[fd].dir_index = dir_index;
62         ProcessTable[pid].fd_table[fd].offset = 0;
63         ProcessTable[pid].fd_table[fd].mode = mode;
64
65         printf("open success: pid=%d, file=%s, fd=%d\n",
66             pid, filename, fd);
67         return fd;
68     }
69 }
70
71 //fd分配失败,回退已经做的修改
72 if (mode == MODE_READ) {
73     pthread_mutex_lock(&Directory[dir_index].mutex);
74     Directory[dir_index].reader_count--;
75     if (Directory[dir_index].reader_count == 0) {
76         sem_post(&Directory[dir_index].rw_lock); // 最后一个读者释放文件
77     }
78     pthread_mutex_unlock(&Directory[dir_index].mutex);
79 } else {
80     sem_post(&Directory[dir_index].rw_lock); // 写者释放文件
81 }
82
83 printf("open failed: fd table full\n");
84 return -1;
85 }
86

```

整体实现思路类似任务一, 在 Directory 数组中查找要打开的文件时前后加锁保护, 确保一致性。之后按读者-写者问题的经典解法实现: 当以只读模式 (MODE_READ) 打开时, 该进程视为读者, 需要先互斥修改 reader_count(通过 pthread_mutex_lock/unlock 对 mutex 加锁/解锁), 若为第一个读者, 还需对资源信号量执行 P 操作 (sem_trywait(&Directory[dir_index].rw_lock)), 以阻塞后续写者; 当以只写模式 (MODE_WRITE) 或读写模式 (MODE_RDWR) 打开时, 该

进程视为写者, 直接对资源信号量执行 P 操作, 确保对文件的互斥访问, 阻止其他读者和写者。这里实现时使用 `sem_trywait` 而非 `sem_wait`, 原因是若使用 `sem_wait` 阻塞等待, 当前交互界面会被卡住, 无法手动切换进程并释放资源。因此本实验采用非阻塞方式, 即文件被占用时直接返回失败。在实际多进程环境中应当使用真正的阻塞等待来实现互斥。对于打开的文件需要分配文件描述符 `fd`, 当 `fd` 表已满时意味着文件打开失败, 此时需要回退前一步做的修改, 即恢复 `reader_count` 以及释放资源信号量 (`sem_post`)。

- 关闭文件

关闭文件函数

```

1  bool fs_close(int pid, int fd)
2  {
3      if (pid < 0 || pid >= MAX_PROCESS || fd < 0 || fd >= MAX_FD) {
4          return false;
5      }
6
7      if (ProcessTable[pid].fd_table[fd].used == 0) {
8          printf("close failed: invalid fd\n");
9          return false;
10     }
11
12     int dir_index = ProcessTable[pid].fd_table[fd].dir_index;
13     int mode = ProcessTable[pid].fd_table[fd].mode;
14
15
16     if (mode == MODE_READ) {
17         pthread_mutex_lock(&Directory[dir_index].mutex);
18         Directory[dir_index].reader_count--;
19         if (Directory[dir_index].reader_count == 0) {
20             sem_post(&Directory[dir_index].rw_lock); // 最后一个读者释放文件
21         }
22         pthread_mutex_unlock(&Directory[dir_index].mutex);
23     }
24     else if (mode == MODE_WRITE || mode == MODE_RDWR) {
25         sem_post(&Directory[dir_index].rw_lock); // 写者释放文件
26     }
27
28     //释放已分配的fd
29     ProcessTable[pid].fd_table[fd].used = 0;
30     ProcessTable[pid].fd_table[fd].dir_index = -1;
31     ProcessTable[pid].fd_table[fd].offset = 0;
32     ProcessTable[pid].fd_table[fd].mode = 0;
33
34     printf("close success: pid=%d, fd=%d\n", pid, fd);
35     return true;
36 }

```


同样采用读者-写者问题的经典解法, 类似打开文件的实现。根据进程数组中对应文件描述符记录的打开模式确定是读者还是写者: 写者 (MODE_WRITE 或 MODE_RDWR) 直接释放资源信号量即可; 读者 (MODE_READ) 需要先互斥减少 reader_count, 当 reader_count 降为 0 时由最后一个读者释放资源信号量。

5.3 任务二函数汇总

下表对任务二实现的所有主要函数进行汇总, 以便对照查阅。

表 2: 任务二主要函数汇总

函数名	功能	实现方法	关键数据结构	同步机制
init_fs()	初始化文件系统	新增信号量/互斥锁初始化, 调用 init_process_table()	Directory[], rw_lock, mutex	信号量初值置 1, 互斥锁初始化
init_process_table()	初始化 PCB 数组	memset 清零, 设置 pid 和 current_dir	ProcessTable[]	无
mkdir(pid,...)	创建目录	互斥锁保护 Directory 数组, 查找空闲项设置属性	Directory[], fs_mutex	互斥锁保护全局目录项
ls(pid,...)	列出当前目录内容	互斥锁保护下遍历 Directory 并打印	Directory[], fs_mutex	互斥锁保护读取一致性
cd(pid,...)	切换当前目录	互斥锁保护下查找目标目录, 更新进程 current_dir	Directory[], ProcessTable[]	互斥锁防止 mkdir 干扰
fs_create(pid,...)	创建文件	互斥锁保护 Directory, 找空闲项设置属性	Directory[], fs_mutex	互斥锁
fs_delete()	删除文件	遍历 Directory 找文件, 沿 FAT 链释放块	Directory[], FAT[], fs_mutex	互斥锁
fs_open(pid,...)	打开文件	互斥锁查找文件 → 读者: 加 reader_count, 首读者 P(rw_lock); 写者: 直接 P(rw_lock) → 分配 fd	Directory[], rw_lock, mutex	读者-写者经典解法, sem_trywait 非阻塞
fs_close(pid,...)	关闭文件	读者: 减 reader_count, 末读者 V(rw_lock); 写者: V(rw_lock) → 清零 fd 表项	Directory[], rw_lock, mutex	读者-写者释放逻辑
fs_read(pid,...)	读取文件内容	沿 FAT 链从 offset 逐块读取到 buffer	Disk[], FAT[], ProcessTable[]	依赖 open 时的 rw_lock 保护
fs_write(pid,...)	写入文件内容	确保 FAT 链有足够块, 从 offset 逐块写入	Disk[], FAT[], ProcessTable[]	依赖 open 时的 rw_lock 保护

5.3.1 文件读写

在 src 文件夹中的 file.c 文件中实现文件的读写、删除等函数, 其实方法类似任务一, 只是在查找目录项、FAT 表时为了确保互斥访问和一致性, 需要前后申请信号量保护。重点修改在删除文件时需要确保没有读者或写者在使用该文件, 否则无法删除, 具体代码如下:

删除文件函数

```

1  bool fs_delete(const char *filename)
2  {
3      pthread_mutex_lock(&fs_mutex);
4
5      for (int i = 0; i < MAX_ENTRY; i++) {
6          if (Directory[i].ac == 1 &&
7              Directory[i].type == 0 &&
8              strcmp(Directory[i].name, filename) == 0) {
9
10             // 检查文件是否正被打开:
11             // reader_count > 0 说明有读者
12             // sem_trywait(&rw_lock) 失败说明有写者
13             pthread_mutex_lock(&Directory[i].mutex);
14             if (Directory[i].reader_count > 0) {
15                 pthread_mutex_unlock(&Directory[i].mutex);
16                 pthread_mutex_unlock(&fs_mutex);
17                 printf("Failed to delete file: %s is in use\n", filename);
18                 return false;
19             }
20         }
21     }
22 }
```

```

20     pthread_mutex_unlock(&Directory[i].mutex);
21
22     if (sem_trywait(&Directory[i].rw_lock) != 0) {
23         pthread_mutex_unlock(&fs_mutex);
24         printf("Failed to delete file: %s is in use\n", filename);
25         return false;
26     }
27     sem_post(&Directory[i].rw_lock); // 成功拿到写锁，立即释放
28
29     int cur = Directory[i].start_block;
30     while (cur != -1) {
31         int next = FAT[cur];
32         FAT[cur] = 0;
33         if (next == -1) break;
34         cur = next;
35     }
36
37     Directory[i].ac = 0;
38     pthread_mutex_unlock(&fs_mutex);
39     printf("File deleted successfully: %s\n", filename);
40     return true;
41 }
42 }
43
44 pthread_mutex_unlock(&fs_mutex);
45 printf("Failed to delete file: %s\n", filename);
46 return false;
47 }

```

如果检查到 reader_count 大于 0, 说明还有读者在访问文件, 无法删除; 如果检查到 sem_trywait(&rw_lock) 返回失败, 说明有写者在访问文件, 同样无法删除。只有当文件空闲没有进程在访问时, 才能删除文件, 具体删除操作于任务一相同, 释放占有的 FAT 表项和 Directory 目录项即可。

5.3.2 文件系统入口

在 src 文件夹的 main.c 文件中实现文件系统入口函数, 实现同样类似任务一, 不同点在于需要实现进程的切换, 这里通过维护当前进程号 pid 的全局变量 current_pid 实现。具体代码如下:

进程切换

```

1 //切换当前操作的进程
2     else if (strcmp(cmd, "pid") == 0) {
3         int new_pid;
4         if (sscanf(line, "%31s %d", cmd, &new_pid) == 2) {
5             if (new_pid >= 0 && new_pid < MAX_PROCESS) {
6                 current_pid = new_pid;
7                 printf("switched to pid %d\n", current_pid);
8             } else {

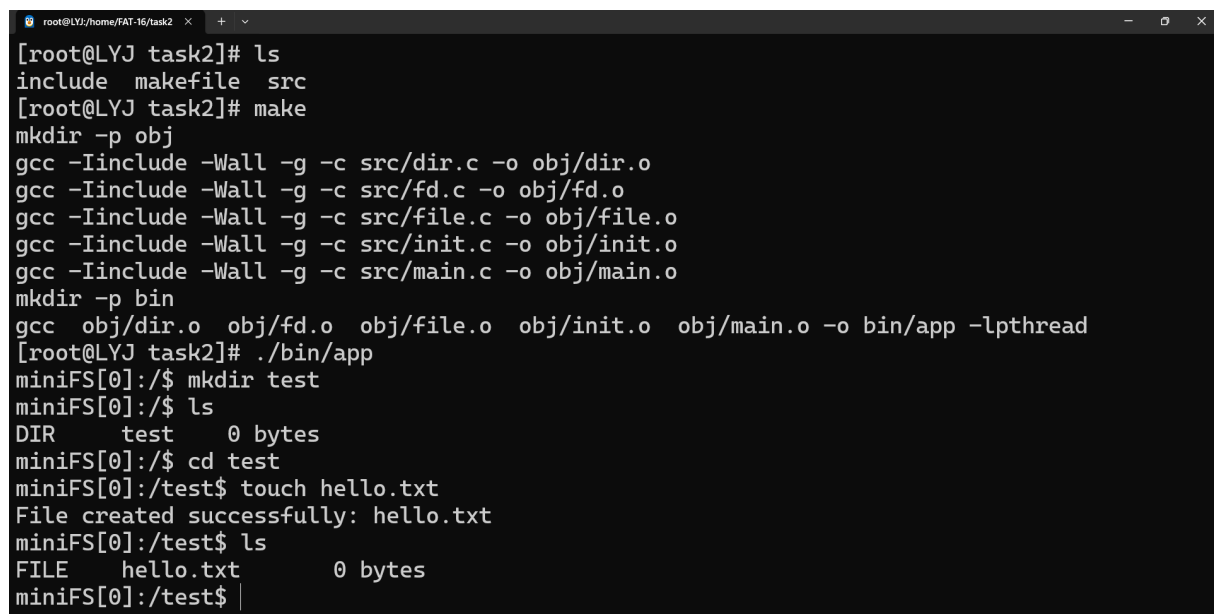
```

```
9         printf("invalid pid (0-%d)\n", MAX_PROCESS - 1);
10     }
11 } else {
12     printf("usage: pid <0-%d>\n", MAX_PROCESS - 1);
13 }
14 }
```

这里当接收到用户 pid n 的命令时会尝试将 current_pid 切换为 n(n 在 0 到 MAX_PROCESS-1 之间视为合法进程号), 此后的操作会基于新的进程进行。其他操作如 mkdir、open 等命令格式与任务一相同, 虽然在函数声明时需要传入 pid 参数, 但这里会自动传入 current_pid, 无需手动传递。

5.3.3 多进程访问测试

上述代码修改完成后即可实现多进程同时访问文件、目录的功能, 代码编译运行结果如下图所示:



```
root@LYJ:/home/FAT-16/task2 x + v
[root@LYJ task2]# ls
include makefile src
[root@LYJ task2]# make
mkdir -p obj
gcc -Iinclude -Wall -g -c src/dir.c -o obj/dir.o
gcc -Iinclude -Wall -g -c src/fd.c -o obj/fd.o
gcc -Iinclude -Wall -g -c src/file.c -o obj/file.o
gcc -Iinclude -Wall -g -c src/init.c -o obj/init.o
gcc -Iinclude -Wall -g -c src/main.c -o obj/main.o
mkdir -p bin
gcc obj/dir.o obj/fd.o obj/file.o obj/init.o obj/main.o -o bin/app -lpthread
[root@LYJ task2]# ./bin/app
miniFS[0]:/$ mkdir test
miniFS[0]:/$ ls
DIR    test    0 bytes
miniFS[0]:/$ cd test
miniFS[0]:/test$ touch hello.txt
File created successfully: hello.txt
miniFS[0]:/test$ ls
FILE    hello.txt    0 bytes
miniFS[0]:/test$ |
```

图 4: 文件系统一般操作

如上图所示, 一般的创建文件夹、创建文件以及切换目录等操作能够正常实现。

```
miniFS[0]:/test$ touch hello.txt
File created successfully: hello.txt
miniFS[0]:/test$ ls
FILE    hello.txt      0 bytes
miniFS[0]:/test$ open hello.txt w
open success: pid=0, file=hello.txt, fd=0
miniFS[0]:/test$ write 0 hello,world!
miniFS[0]:/test$ pid 1
switched to pid 1
miniFS[1]:/$ ls
DIR     test          12 bytes
miniFS[1]:/$ cd test
miniFS[1]:/test$ ls
FILE    hello.txt      12 bytes
miniFS[1]:/test$ open hello.txt r
open failed: file is busy
miniFS[1]:/test$ |
```

图 5: 写进程独占

如上图所示, 当一个写进程 (pid=0) 打开文件时, 切换其他读进程 (pid=1) 尝试打开文件失败, 提示“file is busy”, 说明能实现写进程互斥访问文件的功能。

```
miniFS[1]:/test$ open hello.txt r
open failed: file is busy
miniFS[1]:/test$ pid 0
switched to pid 0
miniFS[0]:/test$ ls
FILE    hello.txt      12 bytes
miniFS[0]:/test$ close 0
close success: pid=0, fd=0
miniFS[0]:/test$ pid 1
switched to pid 1
miniFS[1]:/test$ ls
FILE    hello.txt      12 bytes
miniFS[1]:/test$ open hello.txt r
open success: pid=1, file=hello.txt, fd=0
miniFS[1]:/test$ read 0
hello,world!
miniFS[1]:/test$ pid 2
switched to pid 2
miniFS[2]:/$ cd test
miniFS[2]:/test$ open hello.txt r
open success: pid=2, file=hello.txt, fd=0
miniFS[2]:/test$ read 0
hello,world!
miniFS[2]:/test$ |
```

图 6: 读进程共享

如上图所示, 多个读进程 (pid=1,2) 可以同时访问文件和目录, 读取同一个文件的内容。

```
miniFS[1]:/test$ ls
FILE      hello.txt      12 bytes
miniFS[1]:/test$ open hello.txt r
open success: pid=1, file=hello.txt, fd=0
miniFS[1]:/test$ read 0
hello,world!
miniFS[1]:/test$ pid 2
switched to pid 2
miniFS[2]:/$ cd test
miniFS[2]:/test$ open hello.txt r
open success: pid=2, file=hello.txt, fd=0
miniFS[2]:/test$ read 0
hello,world!
miniFS[2]:/test$ pid 3
switched to pid 3
miniFS[3]:/$ cd test
miniFS[3]:/test$ open hello.txt w
open failed: file is busy
miniFS[3]:/test$
```

图 7: 有读进程时写进程访问失败

如上图所示, 当已经有读进程 (pid=1,2) 打开文件时, 写进程 (pid=3) 无法访问文件, 提示“file is busy”, 说明能实现读进程与写进程互斥访问文件的功能。综上所述, 该文件系统能够实现多进程之间的读者-写者互斥同步, 读写操作均能正确执行, 任务二成功完成。

6 实验问题

6.1 sem_wait 阻塞进程

在任务二实现进程互斥时在一开始使用函数 `sem_wait()` 阻塞进程, 该函数完全符合课上所说的 P 操作, 即将信号量递减后判断是否小于零, 如果是则阻塞当前进程, 否则继续执行。这样在实际运行文件系统时的确能正确实现读写进程间的互斥访问同一文件, 但是一旦阻塞将导致当前交互界面无法接收执行任何命令, 也就无法切换进程、释放资源等操作, 系统正确但不可用。

6.2 使用 sem_trywait 替换

考虑使用 `sem_trywait()` 函数替换 `sem_wait()` 函数, 前者是后者的非阻塞尝试, 成功获取信号量时操作相同, 当失败时不会阻塞进程, 而是立即返回, 这样能实现任务二要求还能保证系统的可用性, 更符合此处实际场景。

7 实验结论

本次实验成功实现了两个任务的预定目标:

任务一完成了基于内存的类 FAT-16 文件系统的设计与实现。该系统以数组 `Disk[DISK_MAXLEN]` 模拟硬盘空间, 在此基础上定义了 FAT 表、目录项 (DirEntry) 和文件描述符 (FileDescriptor) 等核心数据结构, 并实现了目录创建 (mkdir)、目录查看 (ls)、目录切换 (cd)、文件创建 (touch)、文件打开 (open)、文件读取 (read)、文件写入 (write)、文件关闭 (close) 以及文件删除 (rm) 等完整的基本

文件系统操作。经测试验证, 各项功能均能正常运行, 表明 FAT 文件分配表的原理得到了正确的理解与应用。

任务二在任务一的基础上引入多进程并发访问机制, 通过为每个进程维护独立的进程控制块和文件描述符表, 使得多个进程可以同时操作文件系统。针对目录项、FAT 表等全局共享资源, 使用互斥锁 `fs_mutex` 加以保护; 针对普通文件, 采用读者-写者问题的经典解法, 在读进程和写进程之间实现了正确的同步互斥。测试结果表明: 写进程打开文件时其他进程无法访问, 多个读进程可以同时读取同一文件, 有读进程存在时写进程访问会被拒绝, 以上场景均符合预期, 验证了并发控制机制的正确性。

8 收获与心得

通过本次实验, 对操作系统中文件系统的实现原理和并发控制机制有了更深入的理解:

1. 文件系统底层实现的理解。在任务一中, 通过将内存数组模拟为硬盘, 手动实现了文件系统从底层数据结构到上层接口的完整构建过程。FAT 表的链式管理方式直观展示了磁盘空间分配与回收的机制, 目录项结构体揭示了文件元数据管理的基本方法, 文件打开表的引入则体现了操作系统通过缓存机制提升文件访问效率的设计思路。这些实践使课本中抽象的理论概念变得具体形象。

2. 读者-写者问题的工程实现。任务二将经典的读者-写者同步问题应用于实际文件系统场景中。实现过程中需仔细考虑互斥锁和信号量的配合: 用 `mutex` 保护 `reader_count` 的修改、用 `rw_lock` 信号量控制资源访问权限。这加深了对 P/V 操作语义的理解, 尤其是第一个读者和最后一个读者申请和释放资源信号量的逻辑。此外, 使用 `sem_trywait` 替代 `sem_wait` 以解决交互界面响应问题, 也体现了理论算法需结合实际场景灵活调整的工程思维。